

A Small-Scale Extension of CCAC with Improved OOD Classification

RL Course Final Report

Name: _____ Student ID: _____
Name: _____ Student ID: _____
Name: _____ Student ID: _____
Name: _____ Student ID: _____

June 2026

Abstract

Offline safe reinforcement learning aims to learn high-reward policies from a fixed dataset while satisfying safety constraints. Constraint-Conditioned Actor-Critic (CCAC) addresses this problem by conditioning the actor and critics on the remaining safety budget, allowing one trained policy to be evaluated under multiple target costs. This report presents a course-project scale extension of an existing CCAC implementation. The extension replaces the original binary cross-entropy loss for the OOD classifier with focal loss and uses the classifier output as a soft weight in the OOD cost-critic update. The evaluation follows a three-level pipeline. Test A trains only the OOD classifier and measures unsafe/OOD false negatives. Test B trains a simplified cost critic and measures matched-budget OOD-vs-IND cost separation. Test C runs 50k-update full policy training and evaluates reward and realized cost under multiple target costs. The results show stable mechanism-level improvements: the proposed variant substantially reduces false negative rates on BallRun and CarRun and improves matched-budget cost-critic separation on CarRun across two seeds. However, full policy results are mixed. The variant improves reward or strict-budget safety in several BallRun settings, but it fails on BallRun seed 2 at target cost 1 and does not solve CarRun policy safety. Therefore, the FOCAL+SOFT variant should be viewed as a useful diagnostic and small-scale extension rather than a robust new offline safe RL algorithm.

Keywords: offline safe reinforcement learning; CCAC; OOD detection; focal loss; cost critic; constraint-conditioned policy

Member Contributions

The project work was divided into four main parts. The responsible members are left blank for now and can be filled in before submission.

Main responsibilities

No.	Project part	Responsible member
1	Project material collection	_____
2	Code implementation	_____
3	Presentation delivery	_____
4	Experimental report writing	_____

The entries above describe the main stage each student was responsible for. Beyond these primary responsibilities, all group members participated in idea discussion, project iteration,

result interpretation, and final refinement. The overall contribution ratio is therefore considered equal across the four members, namely 25% per member.

1 Introduction

Safe reinforcement learning optimizes task reward while controlling cumulative safety cost. In the offline setting, the learner is restricted to a fixed dataset and cannot interact with the environment to correct high-risk actions. This makes distribution shift especially problematic: when the learned policy selects actions outside the support of the offline data, value estimation can become unreliable and safety constraints may be violated during evaluation.

CCAC is a recent offline safe RL method that learns a constraint-conditioned actor-critic policy [1]. It treats the remaining safety budget κ_t as part of the input to the policy, critics, and generative model. As a result, the same trained policy can be evaluated zero-shot under different target cost thresholds. In the original CCAC design, a conditional variational autoencoder (CVAE) generates budget-conditioned state-action samples, an OOD classifier identifies generated out-of-distribution samples, and the cost critic receives conservative updates on risky generated samples.

This project focuses on a small but safety-relevant issue in the CCAC pipeline. If the OOD classifier misses unsafe or out-of-distribution samples, the cost critic may not receive enough conservative safety signal, and the final policy may remain unsafe. The original implementation trains the classifier with binary cross entropy (BCE) and uses a hard threshold to select OOD samples. A hard OOD decision treats near-threshold samples very differently, for example assigning different behavior to scores 0.49 and 0.51. In safety-critical settings, false negatives are particularly dangerous: an unsafe OOD sample classified as in-distribution may avoid conservative cost regularization.

Motivated by this observation, this report compares original CCAC with one selected variant:

- Baseline: original CCAC, with `classifier_loss=bce` and `ood_mode=hard`.
- Ours: FOCAL+SOFT, with `classifier_loss=focal`, `ood_mode=soft`, `focal_alpha=0.75`, and `focal_gamma=2.0`.

The contribution of this project is threefold. First, it implements two lightweight modifications to CCAC: a focal-loss OOD classifier and a soft OOD weighting rule for the cost-critic update. Second, it evaluates the change through a three-level validation pipeline that separates classifier behavior, cost-critic diagnostics, and final policy performance. Third, it reports both positive and negative evidence. The mechanism-level results are consistently better, but the policy-level results are not uniformly safer.

2 Background

2.1 Offline Safe Reinforcement Learning

Offline safe RL can be formulated as a constrained optimization problem:

$$\max_{\pi} \mathbb{E}_{\pi} [R(\tau)], \quad \text{s.t.} \quad \mathbb{E}_{\pi} [C(\tau)] \leq \epsilon, \quad (1)$$

where $R(\tau)$ is the trajectory return, $C(\tau)$ is the cumulative safety cost, and ϵ is the target cost threshold. Since the training dataset is fixed, the learned policy cannot safely explore the environment. This makes distributional shift and value extrapolation central challenges. The DSRL benchmark [2] provides a suite of offline safe RL tasks and reports reward, cost, and normalized scores for systematic evaluation.

2.2 Review of CCAC

CCAC conditions policy learning on the remaining safety budget. Given budget κ_t , the policy is written as

$$\pi_{\theta}(a_t \mid s_t, \kappa_t), \quad \kappa_{t+1} = \kappa_t - c_t. \quad (2)$$

This design allows one actor to adapt to multiple target cost thresholds at evaluation time.

The algorithm consists of four main components. A CVAE generates budget-conditioned state-action samples. An OOD classifier identifies generated samples that are outside the data distribution. Reward and cost critics estimate return and safety cost. The actor is trained with a Lagrangian-style objective that balances reward maximization and predicted cost control. For safety, the interaction between the OOD classifier and the cost critic is especially important, because conservative cost updates on generated samples shape how the learned policy avoids risky actions outside the dataset support.

3 Method

3.1 Focal OOD Classifier

The original implementation trains the OOD classifier using BCE. This project replaces it with focal loss [3]:

$$\text{FL}(p_t) = \alpha_t(1 - p_t)^{\gamma} \text{BCE}(p, y), \quad (3)$$

where p is the classifier’s predicted OOD probability, $y \in \{0, 1\}$ is the binary label, $p_t = p$ when $y = 1$, and $p_t = 1 - p$ otherwise. The experiment uses $\alpha = 0.75$ and $\gamma = 2.0$. Focal loss down-weights easy examples and gives relatively more weight to hard examples. In this project, the goal is to reduce false negatives, namely unsafe/OOD samples that are incorrectly treated as in-distribution.

3.2 Soft OOD Weighting

The original hard OOD mode uses a threshold:

$$w_{\text{OOD}}(s, a, \kappa) = \mathbb{I}[h_{\psi}(s, a, \kappa) \geq \tau], \quad (4)$$

where h_{ψ} is the OOD classifier and $\tau = 0.5$. The proposed soft mode uses the classifier output directly as a weight:

$$w_{\text{OOD}}(s, a, \kappa) = \text{clamp}(h_{\psi}(s, a, \kappa), 0, 1). \quad (5)$$

This avoids a discontinuous decision around a single threshold. Suspicious samples that do not exceed the hard threshold can still contribute to the OOD cost signal.

3.3 Effect on Cost-Critic Learning

In CCAC, generated OOD samples are used to impose a conservative Lagrangian-style constraint on the cost critic. With soft OOD weighting, the OOD cost loss can be interpreted as

$$\mathcal{L}_{\text{OOD}} = -w_{\text{OOD}}(s, a, \kappa) \lambda_{\phi}(\kappa) (Q_c(s, a, \kappa) - \kappa). \quad (6)$$

Samples with higher OOD scores receive stronger conservative updates. Samples near the threshold are no longer discarded entirely.

4 Experimental Setup

4.1 Tasks and Compared Methods

The experiments use DSRL offline safe RL tasks. The main task is `OfflineBallRun-v0`, which is used for full policy comparison over seeds 0/1/2. The second task is `OfflineCarRun-v0`, which is used for repeated mechanism-level experiments over seeds 0/1 and one seed-0 full-policy check. The CarRun dataset is a locally supplied `SafetyCarRun-v0-40-651.hdf5` file with 130200 transitions. A planned BallCircle fallback task was not included because the public dataset server returned HTTP 502 during the final expansion.

Table 1 summarizes the three-level validation pipeline. Test A and Test B do not replace full policy evaluation. Instead, they isolate whether the classifier and cost critic change in the intended direction before the method is evaluated as a full policy.

Table 1: Experiment protocol and training budget

Test	Task	Seeds	Steps	Purpose
Test A	BallRun, CarRun	BallRun 0; CarRun 0/1	5000	OOD classifier diagnostics, focusing on FNR.
Test B	BallRun, CarRun	BallRun 0; CarRun 0/1	10000	Matched-budget cost-critic separation.
Test C	BallRun	0/1/2	50000	Full policy comparison.
Test C	CarRun	0	50000	Second-task policy-level check.

4.2 Metrics

Test A reports AUROC, AUPRC, false negative rate (FNR), false positive rate (FPR), and expected calibration error (ECE). The most important safety-side metric is FNR, because it measures how often unsafe/OOD samples are incorrectly treated as in-distribution.

Test B reports three cost-critic diagnostics:

- `qc_matched_gap`: mean OOD Q_c minus mean matched IND Q_c ;
- `qc_selected_matched_gap`: mean selected OOD Q_c minus mean matched IND Q_c ;
- `ood_selected_rate`: the fraction of generated samples selected or effectively weighted as OOD.

Test C uses `eval_ccac.py` to evaluate target costs [1, 2, 5, 10], with 20 episodes per target cost. A target is counted as safe if `real_cost` $\leq$ `target_cost`. The “safe targets” column reports how many of the four target costs satisfy this criterion.

4.3 Environment

The experiment records use Python 3.8, PyTorch 2.4.1+cu121, Gymnasium, DSRL, and OSRL. The main GPU is an NVIDIA H100 with 80GB HBM3 memory. Training uses `WANDB_MODE=offline`, and the local HDF5 dataset directory is specified through `DSRL_DATASET_DIR`.

5 Results

5.1 Test A: OOD Classifier

Table 2 shows the classifier-only diagnostics. The FOCAL+SOFT variant substantially reduces FNR in all three settings. On BallRun seed 0, FNR decreases from 0.10380 to 0.03565. On CarRun seed 0, it decreases from 0.13206 to 0.05261. On CarRun seed 1, it decreases from 0.18108 to 0.06185. This supports the first mechanism hypothesis: focal loss makes the classifier miss fewer unsafe/OOD samples.

Table 2: Test A: OOD classifier metrics

Task	Method	Seed	AUROC	AUPRC	FNR	FPR	ECE
BallRun	original	0	0.94270	0.93778	0.10380	0.17205	0.02227
BallRun	FOCAL+SOFT	0	0.93943	0.93326	0.03565	0.28166	0.10002
CarRun	original	0	0.92558	0.88536	0.13206	0.17275	0.03152
CarRun	FOCAL+SOFT	0	0.92649	0.88579	0.05261	0.28111	0.13474
CarRun	original	1	0.92868	0.89186	0.18108	0.12899	0.01671
CarRun	FOCAL+SOFT	1	0.92992	0.89303	0.06185	0.25883	0.12755

The improvement is not free. Compared with original CCAC, FOCAL+SOFT has a higher FPR and worse ECE. This means that the classifier becomes more conservative but less calibrated. Therefore, the correct interpretation is a safety-oriented trade-off rather than a uniformly better classifier.

5.2 Test B: Matched-Budget Cost-Critic Separation

Test B asks whether generated OOD samples receive higher cost estimates under the same query budget. The results are shown in Table 3. On CarRun seed 0 and seed 1, FOCAL+SOFT improves both `qc_matched_gap` and `qc_selected_matched_gap`, suggesting that the more conservative classifier and soft OOD weighting strengthen the OOD cost signal. On BallRun, original CCAC has a negative `qc_matched_gap`, while FOCAL+SOFT produces a positive matched gap. This indicates that the hard-mask baseline can have unstable cost-critic scale in this small experiment.

Table 3: Test B: matched-budget cost-critic separation

Task	Method	Seed	qc_matched_gap	qc_selected_matched_gap	ood_selected_rate
BallRun	original	0	-26.829		0.4186
BallRun	FOCAL+SOFT	0	8.711		0.7555
CarRun	original	0	2.641		0.3106
CarRun	FOCAL+SOFT	0	3.632		0.4868
CarRun	original	1	3.267		0.2267
CarRun	FOCAL+SOFT	1	5.031		0.4255

FOCAL+SOFT also has a higher `ood_selected_rate`, which is consistent with its more conservative classifier behavior in Test A. Overall, Test B supports the second mechanism hypothesis: soft OOD weighting improves matched-budget OOD-vs-IND cost separation.

5.3 Test C: Full Policy Evaluation

The full policy results are more complicated. Table 4 summarizes average reward, average realized cost, and safe targets across the four evaluation budgets.

Table 4: Test C: full policy summary

Task	Method	Seed	Avg reward	Avg real cost	Safe targets
BallRun	original	0	347.274	0.000	4/4
BallRun	FOCAL+SOFT	0	422.771	0.000	4/4
BallRun	original	1	464.176	19.000	3/4
BallRun	FOCAL+SOFT	1	420.406	3.250	4/4
BallRun	original	2	409.096	0.000	4/4
BallRun	FOCAL+SOFT	2	443.294	9.650	3/4
CarRun	original	0	859.456	181.200	0/4
CarRun	FOCAL+SOFT	0	913.301	183.213	0/4

On BallRun, FOCAL+SOFT provides useful but not uniform improvements. In seed 0, it improves reward under all target costs while keeping zero realized cost. In seed 1, it fixes a strict-budget failure: original CCAC has real cost 74 at target cost 1, while FOCAL+SOFT has real cost 0. However, it loses reward at target costs 5 and 10. In seed 2, FOCAL+SOFT improves reward at target costs 2 and 5 while remaining safe there, but it fails badly at target cost 1 with real cost 33.6.

CarRun is the clearest negative policy result. Table 5 shows the key numbers. FOCAL+SOFT obtains higher reward than original CCAC, but realized cost does not improve. Both methods exceed all target cost thresholds by a large margin.

Table 5: Key CarRun policy results

Method	Target 1 cost	Target 5 cost	Avg reward	Safe targets
original	181.00	181.25	859.456	0/4
FOCAL+SOFT	183.05	183.55	913.301	0/4

This result is important because it shows that better mechanism-level OOD diagnostics do not automatically guarantee safer policy behavior.

6 Discussion

The main finding is the gap between mechanism-level improvements and policy-level behavior. Test A and Test B show that FOCAL+SOFT changes the internal OOD safety signal in the intended direction. The classifier misses fewer unsafe/OOD samples, and the cost critic assigns higher cost estimates to OOD samples under matched budgets. This supports the design rationale behind focal loss and soft OOD weighting.

However, the policy-level results are not stable. In some BallRun settings, FOCAL+SOFT increases reward or fixes strict-budget safety failures. In other settings, especially BallRun seed 2 at target cost 1, it introduces a clear safety failure. On CarRun, both original CCAC and FOCAL+SOFT remain unsafe after 50k updates. This suggests that the lightweight OOD modification is not sufficient to solve budget-conditioned safe control on harder tasks.

The three-level evaluation pipeline is therefore useful. If only Test A and Test B were reported, the result would look more positive than it should. If only Test C were reported, it would be difficult to determine whether failures came from the classifier, cost critic, or actor training. Separating mechanism diagnostics from full policy evaluation makes the final conclusion more interpretable.

7 Limitations

This project has several limitations:

- Full policy training uses only 50k updates, which is much smaller than a full benchmark-scale reproduction.
- Policy-level evaluation covers only BallRun seeds 0/1/2 and CarRun seed 0.
- Focal loss reduces FNR but increases FPR and worsens ECE, meaning that classifier calibration becomes worse.
- Soft OOD weighting improves the mechanism-level OOD cost signal, but it does not solve the CarRun policy safety failure.
- The project does not systematically compare against other offline safe RL or offline RL baselines such as CQL [4] and CDT [5].
- The BallCircle fallback task was excluded because the public dataset server failed during the final expansion.

8 Reproducibility Notes

The main experiment scripts are under `CCAC/validation_experiments/scripts/`. Before running experiments, the environment variables `PYTHONPATH`, `WANDB_MODE=offline`, and `DSRL_DATASET_DIR` should be set. The main entry points are:

- `run_test_a_classifier.sh`: classifier-only diagnostics;
- `run_test_b_cost_critic.sh`: cost-critic separation;
- `train_original_ccac_50k.sh`: original CCAC 50k baseline;
- `OSRL/examples/train/train_ccac.py`: full policy training;
- `OSRL/examples/eval/eval_ccac.py`: multi-target-cost policy evaluation.

Full commands, run directories, and raw outputs are recorded in `CCAC/validation_experiments/commands.md` and `CCAC/validation_experiments/results.md`. Report-ready tables are recorded in `CCAC/validation_experiments/final_report_results.md`.

9 Conclusion

This report presents a small-scale extension of CCAC that trains the OOD classifier with focal loss and uses soft OOD scores in the cost-critic update. The proposed variant improves mechanism-level diagnostics: it reduces unsafe OOD false negatives and improves matched-budget cost-critic separation. However, full policy behavior remains mixed. The method improves several BallRun settings but also has a clear strict-budget failure, and it does not solve CarRun safety.

The appropriate conclusion is therefore cautious. FOCAL+SOFT is a useful diagnostic and small-scale extension, not a robust new offline safe RL algorithm. Future work should evaluate the idea on more DSRL tasks, more seeds, longer training budgets, and stronger baselines, while also studying the relationship between classifier calibration, OOD cost regularization strength, and policy stability.

References

- [1] Z. Guo, W. Zhou, S. Wang, and W. Li, “Constraint-conditioned actor-critic for offline safe reinforcement learning,” in *International Conference on Learning Representations*, 2025.
- [2] Z. Liu, Z. Guo, H. Lin, Y. Yao, J. Zhu, Z. Cen, H. Hu, W. Yu, T. Zhang, J. Tan, and D. Zhao, “Datasets and benchmarks for offline safe reinforcement learning,” *Data-centric Machine Learning Research*, 2024.
- [3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal loss for dense object detection,” in *IEEE International Conference on Computer Vision*, pp. 2980–2988, 2017.
- [4] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1179–1191, 2020.
- [5] Z. Liu, Z. Guo, Y. Yao, Z. Cen, W. Yu, T. Zhang, and D. Zhao, “Constrained decision transformer for offline safe reinforcement learning,” in *International Conference on Machine Learning*, vol. 202, pp. 21611–21630, 2023.